# CLASE PRÁCTICA

Seminario de Arquitecturas de Software Distribuidas

## Equipo: LA ESCALONETA

### Integrantes y distribución del trabajo

| Integrante | Grupo | Preguntas |
|---|---|---|
| Marlon R. Cuellar García | C312 | 1 y 2 |
| Raciel Pupo Santos | C312 | 3 |
| Ramón Hernández Camacho | C311 | 4 y documento |
| Leandro Márquez Blanco | C311 | 5 |
| Charly Blanco Sánchez | C311 | 6 |

### Repositorio del proyecto

https://github.com/CuellarMarlon/timeblock/develop

Noviembre de 2026

# Seminario de Arquitecturas de Software Distribuidas
# Ingeniería de Software

## 1. Análisis Comparativo de Comunicación y Acoplamiento

Un sistema de subastas en línea requiere que, al analizar una subasta, se realicen múltiples acciones: notificar al ganador, procesar el pago y actualizar el inventario. En un diseño con Microservicios, un arquitecto podría optar por una coreografía de eventos (comunicación asíncrona) o una orquestación mediante un servicio mediador. En un diseño con Service-Based Architecture, estas acciones probablemente se ejecutarían dentro de un mismo servicio de dominio de grano grueso.

Basado en los conceptos de connascence (connascencia) y los trade-offs de la comunicación síncrona vs. asíncrona, analice:

### ¿Cómo afecta cada enfoque (orquestación vs. coreografía en microservicios, y servicios de grano grueso en SBA) al acoplamiento dinámico entre los componentes del sistema?

Richards y Ford (2020) explican que la connascencia es una forma de medir cuánto dependen dos componentes entre sí. Esta dependencia se manifiesta cuando un cambio en un componente puede obligar a modificar otro para que el sistema siga funcionando correctamente. Los autores distinguen entre connascencia estática, relacionada con las dependencias existentes en el código, y connascencia dinámica, que aparece cuando los componentes interactúan durante la ejecución del sistema.

Un aspecto importante para este análisis es cómo influye el tipo de comunicación entre los componentes. Cuando la comunicación es síncrona, un componente realiza una petición y debe esperar la respuesta del otro. En cambio, cuando es asíncrona, puede enviar un mensaje o evento y continuar trabajando sin esperar a que sea procesado. Esto hace que los componentes dependan menos de que los demás estén disponibles en ese mismo momento (Richards & Ford, 2020).

**Microservicios con orquestación:**

En este enfoque existe un servicio mediador, llamado orquestador, que se encarga de coordinar las operaciones necesarias para completar un proceso. En el sistema de subastas, cuando una subasta finaliza, el orquestador se encargaría de solicitar al servicio de pagos que procese el pago, al de inventario que actualice las existencias y al de notificaciones que informe al ganador.

La orquestación genera una dependencia entre el orquestador y los servicios que coordina, porque necesita conocer qué operaciones ofrece cada uno. Si utiliza comunicación síncrona, esta dependencia es mayor, ya que el orquestador debe esperar las respuestas de los servicios para continuar. Por ejemplo, si el servicio de pagos no está disponible, el proceso podría quedar detenido.

Por tanto, aunque este enfoque facilita controlar el proceso y saber qué debe ejecutarse en cada momento, también concentra la coordinación en un único componente y hace que los servicios dependan de él para completar el flujo de trabajo (Richards & Ford, 2020).

**Microservicios con coreografía de eventos:**

En este enfoque no existe un servicio central que coordine todo el proceso. En su lugar, los servicios se comunican mediante eventos y reaccionan a aquellos que les interesan. Por ejemplo, cuando termina una subasta, el servicio de subastas publica el evento *SubastaFinalizada*. A partir de este evento, los servicios de pagos, inventario y notificaciones realizan sus respectivas tareas de forma independiente.

La coreografía reduce la dependencia directa entre los servicios, especialmente porque normalmente utiliza comunicación asíncrona. Ningún servicio necesita esperar la respuesta de los demás para realizar su trabajo. Por ejemplo, si el servicio de notificaciones deja de funcionar temporalmente, los servicios de pagos e inventario pueden continuar con sus operaciones, mientras el evento queda pendiente para ser procesado posteriormente.

Sin embargo, los servicios siguen dependiendo del formato y significado de los eventos que intercambian. Además, si algunas operaciones deben realizarse en un orden determinado, es necesario controlar esa secuencia. Por tanto, este enfoque reduce la dependencia temporal entre los servicios, pero hace más difícil controlar el proceso completo y gestionar posibles errores (Richards & Ford, 2020).

**Service-Based Architecture (SBA) con servicios de grano grueso:**

En SBA, los servicios suelen agrupar varias funcionalidades que pertenecen a una misma área del negocio. Es decir, en lugar de tener un servicio independiente para cada operación, se reúnen varias operaciones relacionadas dentro de un mismo servicio de dominio (Richards & Ford, 2020).

En nuestro sistema de subastas, podría existir un único servicio encargado de gestionar todo el proceso de finalización de una subasta. Dentro de él se realizarían el procesamiento del pago, la actualización del inventario y la notificación al ganador. De esta manera, no sería necesario que varios servicios independientes se comunicaran entre sí para completar el proceso.

Este enfoque reduce las dependencias entre servicios distribuidos, porque muchas operaciones se realizan dentro del mismo servicio y no necesitan comunicarse a través de la red. También facilita mantener la consistencia de las operaciones, ya que pueden gestionarse mediante transacciones locales.

Sin embargo, las funcionalidades que se encuentran dentro del mismo servicio siguen dependiendo unas de otras. Por ejemplo, un cambio en la lógica de finalización de una subasta podría afectar a otras operaciones relacionadas. Por tanto, aunque se reduce el acoplamiento entre servicios, puede existir un mayor acoplamiento interno entre las funcionalidades que se han agrupado (Richards & Ford, 2020).

En conclusión, cada enfoque gestiona las dependencias de una manera diferente. La orquestación centraliza la coordinación en un servicio mediador, lo que facilita controlar el proceso, pero genera dependencia respecto a este componente. La coreografía permite que los servicios trabajen de forma más independiente mediante eventos asíncronos, reduciendo su dependencia temporal, aunque hace más difícil controlar el proceso completo. Por último, SBA evita muchas comunicaciones entre servicios al agrupar funcionalidades relacionadas, reduciendo el acoplamiento distribuido, pero puede aumentar las dependencias internas.

Por tanto, ninguno de los enfoques elimina completamente el acoplamiento; la diferencia está en dónde se encuentran las dependencias y cómo afectan al funcionamiento del sistema.

## ¿Qué implicaciones tiene cada diseño para la tolerancia a fallos y la consistencia de los datos (ACID vs. BASE) si el servicio de pago falla justo después de que se notifica al ganador?

Para analizar cómo responde cada arquitectura ante un fallo, es importante distinguir entre dos enfoques de consistencia. Por un lado, ACID garantiza que las transacciones se ejecuten correctamente, evitando que una operación quede parcialmente aplicada. Por otro lado, BASE prioriza la disponibilidad del sistema y admite que los datos puedan presentar diferencias temporales entre distintos componentes, hasta alcanzar una consistencia eventual.

Richards y Ford (2020) explican que la comunicación asíncrona permite que los componentes trabajen de forma más independiente, mientras que las interacciones síncronas generan una mayor dependencia entre ellos. Esta diferencia también influye en cómo se gestionan los fallos y la consistencia de los datos.

**Microservicios con orquestación:**

En este enfoque, el orquestador controla el proceso y conoce el estado de las operaciones realizadas por cada servicio. Si el servicio de pagos falla después de notificar al ganador, el orquestador puede detectar el problema e intentar nuevamente el pago o dejar la operación pendiente hasta resolverlo.

La existencia de un coordinador facilita controlar los errores y decidir cómo continuar el proceso. Sin embargo, como cada microservicio gestiona sus propios datos, no es posible garantizar una única transacción ACID que abarque todos los servicios. Para estos casos puede utilizarse el patrón Saga, que divide el proceso en transacciones locales y permite ejecutar acciones compensatorias cuando alguna operación falla (Richards & Ford, 2020).

Por ejemplo, si el pago falla después de actualizar el inventario, el orquestador puede ordenar que se revierta esa actualización o que la venta permanezca pendiente hasta resolver el problema.

**Microservicios con coreografía de eventos:**

En este enfoque, los servicios reaccionan a los eventos que reciben sin depender de un coordinador central. Si el servicio de pagos falla después de que se haya notificado al ganador, los demás servicios pueden haber completado sus operaciones sin conocer ese fallo.

La independencia de los servicios permite que el fallo de uno no detenga necesariamente a los demás. Sin embargo, resulta más difícil controlar el estado general del proceso y coordinar la recuperación de las operaciones.

En este caso, puede producirse una inconsistencia temporal: el ganador ya recibió la notificación, pero el pago todavía no se ha procesado. Esto se relaciona con BASE y la consistencia eventual, ya que los servicios pueden tener información diferente durante un tiempo hasta que se resuelva la operación.

Para gestionar estas situaciones se necesitan mecanismos como reintentos, eventos de compensación y operaciones idempotentes, que permitan procesar nuevamente un evento sin duplicar sus efectos.

**Service-Based Architecture (SBA) con servicios de grano grueso:**

En SBA, las funcionalidades relacionadas se agrupan dentro de un mismo servicio de dominio. En nuestro ejemplo, el procesamiento del pago, la actualización del inventario y la gestión de la subasta podrían realizarse dentro de ese mismo servicio.

Este enfoque facilita mantener la consistencia mediante transacciones ACID, ya que las distintas operaciones pueden ejecutarse dentro de una misma transacción local. Por ejemplo, si el procesamiento del pago falla antes de confirmar la transacción, se pueden revertir los cambios realizados, evitando que la venta quede registrada como completada o que el inventario se actualice incorrectamente.

Sin embargo, al agrupar varias funcionalidades, un fallo que afecte al servicio puede dejar temporalmente indisponibles todas ellas. Además, si el pago depende de una plataforma externa, no queda protegido automáticamente por la transacción local y requiere mecanismos adicionales para gestionar posibles errores.

En conclusión, cada enfoque presenta diferentes ventajas y dificultades para gestionar los fallos y mantener la consistencia. La orquestación facilita controlar el proceso y recuperar operaciones fallidas mediante mecanismos como las Sagas. La coreografía permite que los servicios continúen funcionando independientemente, pero puede generar inconsistencias temporales y dificulta la coordinación de la recuperación. Por último, SBA facilita mantener transacciones ACID al agrupar funcionalidades dentro de un mismo servicio, aunque un fallo puede afectar a varias de ellas.

Por tanto, los microservicios suelen necesitar mecanismos de consistencia eventual y transacciones compensatorias, mientras que SBA facilita mantener la consistencia ACID dentro de los límites de sus servicios. Esto no significa que una arquitectura utilice exclusivamente ACID o BASE, sino que cada enfoque presenta distintas facilidades para mantener la consistencia de los datos.

## 2. Diseño para Elasticidad Extrema y Consistencia de Datos

Considere el sistema de venta de entradas para conciertos descrito en el libro (Capítulo 15). Durante el lanzamiento de entradas para un artista popular, el sistema experimenta un aumento repentino de usuarios de cientos a decenas de miles en segundos. Se le pide como arquitecto diseñar la solución.

Justifique por qué una Space-Based Architecture sería una elección más adecuada que una Microservices Architecture para este escenario específico, enfocándose en las características de elasticidad y escalabilidad.

Sin embargo, la Space-Based Architecture introduce el problema de las colisiones de datos en cachés replicadas. Describa cómo un arquitecto podría mitigar el riesgo de colisiones de datos en el contexto de la venta de asientos numerados, donde la consistencia es crucial. ¿Qué mecanismos o trade-offs consideraría?

### ¿Por qué elegir Space-Based Architecture frente a Microservices Architecture?

Richards y Ford (2020, capítulo 15) presentan la Space-Based Architecture como una arquitectura especialmente adecuada para sistemas que necesitan soportar grandes cantidades de usuarios y cambios repentinos en la demanda. Su funcionamiento se basa en distribuir el procesamiento y los datos entre diferentes unidades de ejecución, evitando depender de una única base de datos central que pueda convertirse en un cuello de botella.

En nuestro caso, durante el lanzamiento de entradas para un artista popular, el sistema podría pasar de cientos a decenas de miles de usuarios en cuestión de segundos. Aunque una arquitectura de microservicios permite escalar servicios de manera independiente, podría tener problemas si todos dependen de una misma base de datos central, ya que esta podría saturarse ante tantas solicitudes simultáneas.

En cambio, Space-Based Architecture permite distribuir las solicitudes entre diferentes unidades de procesamiento y añadir nuevas unidades cuando aumenta la demanda. Además, utiliza datos en memoria para reducir los tiempos de respuesta y evitar que todas las operaciones dependan de una base de datos central.

Por tanto, este enfoque resulta más adecuado para el escenario porque ofrece mayor capacidad para absorber aumentos repentinos de tráfico y distribuir la carga, facilitando la elasticidad y la escalabilidad del sistema.

### Riesgo de colisiones de datos en cachés replicadas

En Space-Based Architecture, los datos pueden mantenerse replicados en diferentes unidades de procesamiento. Esto mejora el rendimiento, pero también puede provocar problemas cuando varias unidades modifican el mismo dato al mismo tiempo.

Richards y Ford (2020) explican que estas situaciones pueden provocar colisiones de datos, especialmente cuando dos unidades trabajan con copias diferentes de una misma información y posteriormente intentan sincronizar sus cambios.

En nuestro sistema de venta de entradas, esto podría ocurrir si dos usuarios intentan comprar el mismo asiento. Ambos podrían consultar su disponibilidad desde unidades diferentes y recibir como respuesta que está libre. Si los dos realizan la compra antes de que se actualicen las réplicas, el mismo asiento podría venderse dos veces.

### Posibles soluciones

Para evitar este problema, utilizaría las siguientes medidas:

- **Caché distribuida:** mantener la disponibilidad de los asientos en un espacio de datos compartido, en lugar de permitir que cada unidad gestione una copia independiente. Esto reduce el riesgo de que existan diferentes versiones de la disponibilidad de un mismo asiento, aunque puede aumentar la latencia de las operaciones.
- **Operaciones atómicas:** al reservar un asiento, el sistema debe comprobar que sigue disponible y cambiar su estado a reservado en una única operación. Así, si dos usuarios intentan reservarlo simultáneamente, solamente uno podrá conseguirlo.
- **Bloqueos temporales:** cuando un usuario selecciona un asiento, se puede reservar durante un tiempo limitado para que complete el pago. Si no termina la operación dentro de ese tiempo, el asiento vuelve a estar disponible.

Por tanto, combinaría las cachés replicadas para información que no necesita actualizarse constantemente, como los datos del concierto, con una caché distribuida y operaciones atómicas para gestionar la disponibilidad de los asientos. De esta manera, se mantiene un buen rendimiento sin poner en riesgo la consistencia de las reservas.

En conclusión, Space-Based Architecture resulta adecuada para este escenario porque permite distribuir la carga y añadir capacidad de procesamiento cuando aumenta repentinamente el número de usuarios. Sin embargo, las cachés replicadas pueden provocar inconsistencias cuando varias unidades modifican simultáneamente los mismos datos.

Por ello, es importante elegir cómo almacenar y gestionar cada tipo de información. En particular, para los asientos numerados debe priorizarse la consistencia sobre el rendimiento, garantizando que un mismo asiento no pueda reservarse o venderse más de una vez.

## 3. Análisis de fallos y resiliencia en arquitecturas dirigidas por eventos

### 3.1 Impacto del fallo en el flujo del sistema y en el usuario final

En la topología de *broker* no existe un componente central que controle el flujo de trabajo. Cada procesador de eventos reacciona a un evento, realiza su tarea y publica un nuevo evento anunciando lo que hizo, sin saber quién lo consumirá (Richards & Ford, 2020, cap. 14). Esa ausencia de coordinador da a la topología su alto desacoplamiento, escalabilidad y rendimiento.

A cambio, los autores enumeran como desventajas el control del flujo de trabajo, el manejo de errores, la recuperabilidad, la capacidad de reinicio y la inconsistencia de datos (Richards & Ford, 2020, cap. 14, tabla 14-1).

Cuando un procesador falla repetidamente con un tipo concreto de mensaje por un error en los datos, el impacto se manifiesta en varios niveles:

1. **Nadie se entera del fallo y el proceso de negocio se detiene.** Como ningún mediador supervisa la transacción de negocio, ningún otro componente sabe que el procesador falló. El proceso queda atascado y no avanza sin intervención automática o manual. Mientras tanto, los demás procesadores siguen trabajando como si todo fuera bien. En el ejemplo del libro, el procesador de inventario descuenta las *existencias* aunque el pago no se haya completado (Richards & Ford, 2020, cap. 14).

2. **Inconsistencia de datos.** El procesador que falla nunca publica su evento de salida, por lo que los procesadores que dependen de él nunca se activan. Los que reaccionaron en paralelo al evento inicial sí completaron su parte. El resultado es un estado parcial, por *ejemplo* un pedido con el inventario reservado pero sin cobrar ni enviar.

3. **No se puede reiniciar la transacción.** Como otras acciones ya se ejecutaron de forma asíncrona, no es posible volver a enviar el evento inicial: se duplicarían. Ningún componente conoce ni es dueño del estado de la petición original, así que nadie sabe dónde se quedó el proceso (Richards & Ford, 2020, cap. 14).

4. **Mensaje "envenenado" y degradación de la cola.** Si el procesador no confirma el mensaje defectuoso, el broker lo vuelve a entregar indefinidamente. Como el error está en los datos y no en una condición transitoria, ningún reintento tendrá éxito. El procesador se queda atascado y no lee los siguientes mensajes, lo que afecta a la capacidad de respuesta de todos los que esperan en la cola. Si, por el contrario, confirma el mensaje y se limita a registrar el error, el evento se pierde.

Para el **usuario final** el efecto es especialmente engañoso. En una arquitectura asíncrona el usuario recibe una confirmación inmediata ("su solicitud fue recibida") con la promesa de que la *acción se completará después*. Cuando el fallo ocurre, no hay un usuario al que responder de forma síncrona para que corrija el error (Richards & Ford, 2020, cap. 14). El usuario no recibe ningún error, pero la acción nunca se completa o se completa a medias. Lo descubre más tarde (un envío que no llega, un saldo incorrecto), lo que genera reclamaciones, pérdida de confianza y trabajo manual de soporte.

## 3.2 Solución con el Workflow Event Pattern

El patrón de eventos de flujo de trabajo (*Workflow Event Pattern*) es un patrón de arquitectura reactiva que aborda a la vez la resiliencia y la capacidad de respuesta. El sistema maneja los errores sin dejar de procesar mensajes, mediante **delegación, contención y reparación** a través

de un delegado de flujo de trabajo (Richards & Ford, 2020, cap. 14). Su funcionamiento es el siguiente:

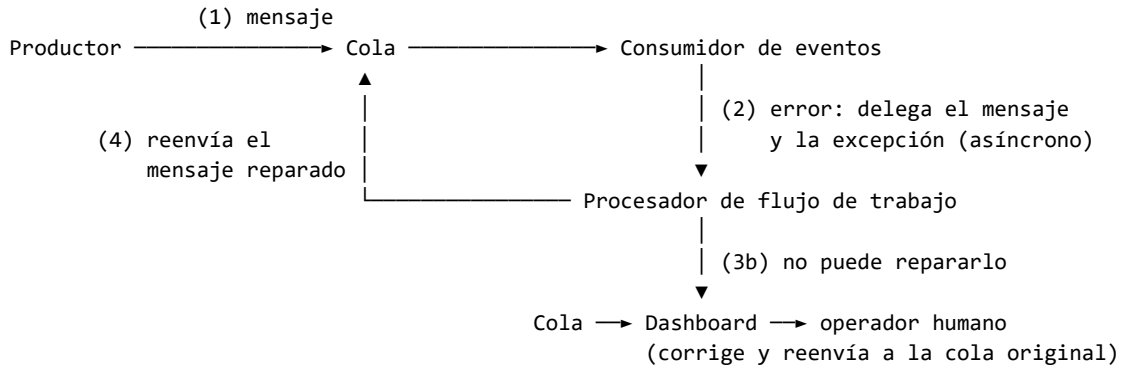



*Figura 1. Flujo del Workflow Event Pattern (elaboración propia a partir de Richards & Ford, 2020, cap. 14).*

1. **Consumidor de eventos.** Si encuentra un error al procesar un mensaje, delega inmediatamente el error, de forma asíncrona, al procesador de flujo de trabajo junto con la información de la excepción. Después pasa al siguiente mensaje de la cola. No dedica tiempo a diagnosticar el error, de modo que la capacidad de respuesta del resto de mensajes no se ve afectada.

2. **Procesador de flujo de trabajo (workflow processor).** Analiza qué está mal en el mensaje. Puede tratarse de un error estático y determinista, o puede recurrir a algoritmos de aprendizaje automático para detectar anomalías en los datos. En cualquier caso, intenta reparar los datos de forma programática, sin intervención humana.

3. **Reenvío o escalado.**

- (3a) Si consigue repararlo, lo reenvía a la cola original. El consumidor lo trata como un mensaje nuevo.
- (3b) Si no puede determinar el problema, lo envía a otra cola de la que lo recibe un *dashboard*: una aplicación parecida a un cliente de correo donde una persona responsable revisa el mensaje, lo corrige manualmente y lo reenvía a la cola original, normalmente mediante la cabecera *reply-to* del mensaje (Richards & Ford, 2020, cap. 14).

4. **Reprocesamiento.** El consumidor procesa el mensaje corregido y publica su evento de salida, con lo que los procesadores dependientes se activan normalmente.

El libro lo ilustra con una firma que recibe lotes de órdenes bursátiles de un asesor externo. Una orden trae el texto "SHARES" pegado al número de acciones, lo que provoca un error de conversión. El servicio de colocación de órdenes delega ese error a un servicio de errores, que elimina el texto sobrante y reenvía la orden, mientras el resto del lote se sigue procesando (Richards & Ford, 2020, cap. 14).

**Por qué mitiga el problema.**

- Los reintentos inútiles desaparecen y la cola no se atasca.

- Ningún mensaje defectuoso se pierde en silencio: todos tienen un camino de recuperación, sea automático o manual.
- El dashboard hace visibles los fallos, algo que la topología de broker no ofrece por sí misma.

A diferencia de un simple canal de mensajes muertos (*dead letter channel*), que solo aparta el mensaje que no se pudo entregar (Hohpe & Woolf, 2003), el patrón añade la reparación y el reenvío, cerrando el ciclo.

**Componentes y medidas adicionales necesarios:**

- Un canal asíncrono de errores entre el consumidor y el procesador de flujo de trabajo.
- El propio procesador de flujo de trabajo, con sus reglas de reparación.
- La cola y la aplicación del dashboard, con personal responsable de atenderla. Por seguridad conviene que registre quién modificó cada mensaje (auditoría).
- Un contador de intentos por mensaje, para que uno que vuelva a fallar tras la reparación no entre en un ciclo infinito.
- Técnicas para no perder mensajes en el trayecto (Richards & Ford, 2020, cap. 14):
    - colas persistentes con envío síncrono (*synchronous send*);
    - modo de confirmación del cliente (*client acknowledge mode*), para que un mensaje no se elimine de la cola si el consumidor cae antes de procesarlo;
    - transacciones ACID con *last participant support* al persistir.
- Alertas cuando crezca el volumen de mensajes reparados. Si muchos fallan por la misma causa, el error es sistémico (por ejemplo, un productor que cambió su formato) y debe corregirse en el origen, en lugar de repararse mensaje a mensaje.

**Trade-offs que introduce:**

- **Procesamiento fuera de orden.** Es la principal consecuencia: los mensajes con error se procesan fuera de secuencia al reenviarse. En el ejemplo bursátil importa, porque las órdenes de una misma cuenta deben ejecutarse en orden (una venta de IBM antes de una compra de AAPL). Mantener el orden dentro de un contexto es posible pero complejo. El libro propone que el consumidor registre el número de cuenta de la orden con error y guarde en una cola temporal, en orden FIFO, todas las órdenes posteriores de esa cuenta. Cuando la orden reparada se procesa, extrae y procesa las pendientes en orden (Richards & Ford, 2020, cap. 14). Esto añade estado y lógica al consumidor.
- **Idempotencia.** Si el consumidor llegó a ejecutar parte del trabajo antes de fallar, el reenvío puede duplicar efectos. Los consumidores deben ser idempotentes.
- **Latencia de recuperación.** Un mensaje que requiere corrección manual puede tardar minutos u horas en completarse.
- **Más complejidad y coste operativo.** Hay más componentes que desplegar y monitorizar, y personal que atienda el dashboard.

- **Riesgo de los cambios manuales.** Una corrección humana puede introducir errores nuevos, de ahí la necesidad de validación y auditoría.

En resumen, el patrón cambia orden garantizado y simplicidad por capacidad de respuesta y recuperabilidad: el sistema sigue procesando y ningún fallo se pierde, a costa de gestionar mensajes fuera de orden y de añadir infraestructura.

## 4. El dilema de la granularidad y el acoplamiento en microservicios

### 4.1 Error fundamental en el diseño de los servicios

El error fue descomponer el sistema por entidades de datos (Empleado, Departamento, Nómina, Beneficios) en lugar de por contextos acotados de negocio. Richards y Ford llaman a este antipatrón la **trampa de la entidad** (*entity trap*). Surge cuando el arquitecto toma erróneamente las relaciones de la base de datos como si fueran los flujos de trabajo de la aplicación, una correspondencia que rara vez se da en la realidad, y suele indicar que no se pensó en los flujos reales del negocio (Richards & Ford, 2020, cap. 8). Al tratar el aislamiento de datos en microservicios, los autores advierten expresamente que no hay que modelar los servicios como si fueran entidades individuales de una base de datos (Richards & Ford, 2020, cap. 17).

Según el capítulo 17, el propósito de los límites de un microservicio es capturar un dominio o un flujo de trabajo (*bounded context*), con su comportamiento y sus datos (Evans, 2003; Richards & Ford, 2020, cap. 17). "Contratar a un empleado" es un flujo de negocio que no pertenece a ninguna de las cuatro entidades, sino que las atraviesa. Repartirlo entre cuatro servicios provoca:

- **Servicios demasiado finos.** Los autores señalan que el error frecuente es hacer servicios tan pequeños que necesitan comunicarse entre sí para realizar trabajo útil. Recuerdan, citando a Martin Fowler, que el término "microservicio" es una etiqueta y no una descripción (Richards & Ford, 2020, cap. 17).
- **Transacciones distribuidas.** La contratación debería ser atómica: no tiene sentido un empleado sin departamento o sin nómina. Pero sus datos están en cuatro bases de datos. Construir transacciones entre servicios viola el principio de desacoplamiento y crea la peor forma de *connascence* dinámica, la connascencia de valor. El consejo del libro es directo: no hacer transacciones en microservicios, sino corregir la granularidad, porque los límites transaccionales son uno de los indicadores habituales de la granularidad correcta (Richards & Ford, 2020, cap. 17).
- **Acoplamiento dinámico alto.** La regla de localidad de la connascence, formulada por Jim Weirich y recogida por Richards y Ford, dice que cuanto más distantes estén los elementos, más débiles deben ser las formas de connascence entre ellos (Richards & Ford, 2020, cap. 3). Aquí ocurre lo contrario: formas fuertes de connascence (de ejecución, de tiempo, de valor) entre servicios desplegados por separado.
- **Un solo cuántum en la práctica.** Las llamadas síncronas crean connascence dinámica durante la llamada: mientras uno espera al otro, ambos deben tener las mismas características

operativas, como escalabilidad y disponibilidad (Richards & Ford, 2020, cap. 7). Durante la contratación, los cuatro servicios se comportan como un único *architecture quantum*.

- **Fragilidad multiplicada.** La disponibilidad de la operación es el producto de la de cada servicio: con un 99 % cada uno, la contratación queda en $0{,}99^4 \approx 96$ %. Además, cualquier cambio en el contrato de un servicio obliga a coordinarse con los demás.

El resultado es un **monolito distribuido**: conserva el acoplamiento del sistema original y además suma la latencia, los fallos parciales y la complejidad operativa de un sistema distribuido.

## 4.2 Nueva estrategia de descomposición

La redefinición de límites se apoya en las tres guías de granularidad del capítulo 17 (Richards & Ford, 2020):

- **Propósito.** Cada microservicio debe tener una cohesión funcional muy alta y aportar un comportamiento significativo a la aplicación.
- **Transacciones.** Los contextos acotados son flujos de negocio, y las entidades que deben cooperar en una transacción suelen señalar un buen límite de servicio.
- **Coreografía.** Si un conjunto de servicios bien aislados necesita mucha comunicación para funcionar, conviene volver a agruparlos en un servicio mayor para evitar esa sobrecarga.

El mismo capítulo reconoce la excepción contraria: dos partes que necesitan características arquitectónicas muy distintas justifican límites de servicio separados (Richards & Ford, 2020, cap. 17). Esto concuerda con la idea de cuántum: partes con necesidades operativas diferentes deben poder desplegarse y escalar por separado (Richards & Ford, 2020, cap. 7).

**Agrupación propuesta para la operación de contratación:**

| Servicio | Agrupa | Justificación |
|---|---|---|
| **Gestión de Personal** | Empleado, Departamento, puesto y contrato | **Guía de transacciones.** Contratar exige crear el empleado, asignarlo a un departamento y registrar su contrato de forma atómica. En un solo servicio con su propia base de datos, esto es una transacción ACID local. |
| **Compensación** | Nómina y Beneficios | **Guía de coreografía.** El cálculo de la nómina necesita las deducciones de los beneficios en cada período, y ambos manejan los mismos datos (salario, cuenta bancaria); separados, cada cierre exigiría mucha comunicación entre ellos. **Características distintas frente a Personal.** Mayores requisitos de seguridad y cumplimiento, |

| | | |
|---|---|---|
| | | picos de carga propios (cierre de mes) y cambios frecuentes por la normativa fiscal. |

**Flujo de contratación rediseñado:**

1. El cliente invoca la operación "contratar" en Gestión de Personal, que crea el empleado, la asignación y el contrato en una sola transacción local.

2. En esa misma transacción registra el evento *EmpleadoContratado*, que después se publica en el broker. El patrón *transactional outbox* (Richardson, 2018) garantiza que el evento se publique si y solo si la transacción se confirma.

3. Compensación consume el evento mediante coreografía, sin mediador, que es el estilo natural de comunicación en microservicios (Richards & Ford, 2020, cap. 17). Con él crea el perfil de nómina y la inscripción por defecto en los beneficios. Los datos del empleado que necesita (identificador, nombre, centro de costo) los mantiene como una copia de solo lectura, replicada a partir de los eventos. Es una de las opciones que el libro plantea frente a una única fuente de verdad: replicar o cachear la información en lugar de consultarla cada vez (Richards & Ford, 2020, cap. 17).

4. Si Compensación está caída, el evento espera en la cola y se procesa cuando se recupere. Para ello el consumidor debe ser idempotente, usando como clave el identificador del empleado.

La relación entre Personal y Compensación pasa a ser de **consistencia eventual**. Es aceptable porque el negocio no necesita el perfil de nómina en el instante de la contratación, sino antes del primer cierre de pago.

**Comparación:**

| **Aspecto** | **Diseño por entidades** | **Diseño por contextos** |
|---|---|---|
| **Llamadas para contratar** | Cadena síncrona entre los 4 servicios | 1 transacción local + 1 evento asíncrono |
| **Atomicidad** | Transacción distribuida (o inconsistencias) | ACID dentro de Gestión de Personal |
| **Disponibilidad de la operación** | Producto de las 4 disponibilidades | La de Gestión de Personal |
| **Acoplamiento dinámico** | Síncrono, en cadena | Asíncrono, por un único evento |

| | | |
|---|---|---|
| **Despliegue** | Coordinado entre servicios | Independiente por servicio |

**Trade-offs aceptados:**

- Gestión de Personal es un servicio más grande, con más código y una base de datos más amplia.
- La nómina se crea con un pequeño retraso, lo que exige vigilar los eventos no procesados.
- Hay datos replicados que deben mantenerse sincronizados.

Si el proceso de incorporación crece (alta de cuentas informáticas, entrega de equipos, formación obligatoria), la coreografía puede degenerar en un *front controller*: el primer servicio acaba actuando como mediador además de cumplir su responsabilidad de dominio. En ese caso es preferible crear un mediador local que concentre la coordinación de ese flujo (Richards & Ford, 2020, cap. 17).

## 5. Comparación de atributos de calidad y trade-offs

### 5.1 Service-Based vs. Microservices frente a los tres requisitos

| Requisito | Service-Based Architecture | Microservices Architecture |
|---|---|---|
| **1. Consistencia ACID absoluta** | Entre 4 y 12 servicios de dominio de grano grueso que suelen compartir una base de datos. Como cada servicio abarca un dominio completo, las transacciones de negocio usan transacciones ACID normales, con *commit* y *rollback*, dentro de un único servicio (Richards & Ford, 2020, cap. 13). | Cada servicio tiene su propia base de datos, por lo que una operación que abarque varios servicios (cuentas, libro mayor, antifraude) se apoya en transacciones BASE, con consistencia eventual (Richards & Ford, 2020, cap. 13). Para coordinarlas se usa el patrón *saga*: un mediador ejecuta transacciones compensatorias, con estados pendientes y mucho tráfico de coordinación. Los autores recomiendan usarlo con moderación (Richards & Ford, 2020, cap. 17). |
| **2. Despliegue independiente y rápido** | Los servicios de dominio se despliegan por separado, con más frecuencia y menos riesgo que un monolito. Sin embargo, cada despliegue abarca un dominio completo, y la base de datos | Es uno de sus puntos fuertes: unidades de despliegue mínimas y fuerte apoyo al despliegue automatizado. Según los autores, los microservicios no podrían existir sin DevOps y la automatización |

| | | |
|---|---|---|
| | compartida acopla a los servicios a través del esquema (Richards & Ford, 2020, cap. 13). | de las tareas operativas (Richards & Ford, 2020, cap. 17). |
| **3. Equipo con experiencia mixta** | La simplicidad y el coste son factores que la distinguen: es uno de los estilos distribuidos más fáciles y económicos de implementar (Richards & Ford, 2020, cap. 13). Resulta asumible por un equipo heterogéneo. | Exige dominar sistemas distribuidos, sagas, idempotencia, trazabilidad y automatización operativa. Con un equipo mixto aumenta el riesgo de caer en servicios demasiado finos y en un monolito distribuido, como en la pregunta 4. |

**Conclusión:** la **Service-Based Architecture** es la más adecuada para cumplir el requisito de consistencia de datos. Richards y Ford afirman que preserva las transacciones ACID mejor que cualquier otra arquitectura distribuida gracias a la granularidad gruesa de sus servicios: en la mayoría de los casos la transacción queda dentro de un único servicio de dominio, con el *commit* y *rollback* tradicionales de un monolito (Richards & Ford, 2020, cap. 13).

Hay una salvedad que el CTO debe conocer. Si la interfaz de usuario o el API gateway orquestan dos o más servicios de dominio, esa transacción también dependería de sagas y de transacciones BASE (Richards & Ford, 2020, cap. 13). Por eso las operaciones financieras deben diseñarse para que cada una quede dentro de un único dominio.

En microservicios, la consistencia absoluta solo se logra si toda la transacción cabe en un servicio, y ampliar los servicios para conseguirlo acerca el diseño a una arquitectura basada en servicios. Si además se tiene en cuenta el tercer requisito, la recomendación global sigue siendo Service-Based.

## 5.2 Impacto sobre el despliegue independiente y trade-offs del CTO

En **Service-Based Architecture**, el despliegue independiente existe, pero es más grueso y depende de los datos:

- **Unidad de despliegue grande.** Un cambio en la colocación de pedidos dentro de un *OrderService* obliga a probar todo el servicio, incluido el procesamiento de pagos. Además, al desplegar más código hay más riesgo de romper algo. En microservicios, el mismo cambio solo afectaría a un pequeño servicio *OrderPlacement* (Richards & Ford, 2020, cap. 13). En el sistema financiero, un cambio en "Transferencias" implicaría redesplegar y volver a probar todo el dominio de cuentas.
- **Acoplamiento por el esquema.** Si no se gestiona bien, cambiar una tabla compartida puede afectar a muchos servicios y salir muy caro. La peor práctica es una única biblioteca compartida con todos los objetos de entidad, porque cualquier cambio de tablas obliga a modificar y redesplegar todos los servicios. El libro propone particionar lógicamente la base

de datos y reflejar esa partición en bibliotecas compartidas federadas por dominio, de modo que un cambio solo afecte a los servicios que usan esa partición. Las entidades comunes a todos quedan bajo control del equipo de base de datos (Richards & Ford, 2020, cap. 13).

- **Un solo cuántum arquitectónico.** Si todos los servicios comparten la misma base de datos, el sistema completo es un único cuántum aunque tenga varios servicios desplegados por separado. Federar la base de datos permite tener varios cuantos (Richards & Ford, 2020, caps. 7 y 13).

En **Microservices Architecture**, cada servicio se despliega con total autonomía. A cambio, el estilo exige automatización de despliegue, observabilidad y gestión de contratos entre servicios, es decir, la madurez de DevOps que el libro considera su requisito previo (Richards & Ford, 2020, cap. 17).

**Trade-offs que debe aceptar el CTO si prioriza la consistencia sobre la agilidad de despliegue:**

1. **Despliegues de mayor alcance.** Cada entrega toca un servicio de dominio completo, exige más pruebas de regresión y tiene más riesgo de romper funcionalidad no relacionada con el cambio.

2. **Coordinación por la base de datos.** Los cambios de esquema necesitan gobierno y acuerdo entre equipos, lo que reduce su autonomía frente a microservicios.

3. **Menor escalabilidad y elasticidad.** El libro valora la escalabilidad de este estilo con tres estrellas y la elasticidad con dos, porque al ser servicios de grano grueso se replica más funcionalidad de la necesaria (Richards & Ford, 2020, cap. 13). La base de datos compartida es además un posible cuello de botella.

4. **Un punto común de fallo.** Los servicios de dominio no se llaman entre sí, de modo que la caída de uno no afecta a los demás (Richards & Ford, 2020, cap. 13). En cambio, una degradación de la base de datos compartida sí los afecta a todos.

5. **Menor capacidad evolutiva.** Si el negocio crece mucho, quizá haya que migrar más adelante algunas partes a un estilo más distribuido, con el coste que eso supone.

**Mitigaciones recomendadas:**

- **Arquitectura híbrida.** Mantener en Service-Based el núcleo transaccional (cuentas, transferencias, libro mayor) y separar como servicios independientes las capacidades periféricas que toleran consistencia eventual, como notificaciones, reportes o el análisis de fraude asíncrono. Así se gana agilidad donde la consistencia no es crítica.
- **Funciones de aptitud (fitness functions).** Son mecanismos que evalúan de forma objetiva una característica arquitectónica, por ejemplo con herramientas como ArchUnit (Richards & Ford, 2020, cap. 6). Integradas en el *pipeline*, pueden comprobar en cada entrega que ningún servicio accede a las tablas de otro dominio y que no aparecen dependencias prohibidas entre módulos.

En síntesis, el CTO cambia agilidad de despliegue y escalabilidad por integridad transaccional y simplicidad. En un sistema financiero, un error de consistencia (dinero duplicado o perdido) cuesta mucho más que una entrega algo más lenta, así que el intercambio está justificado.

# 6. El Legado de SOA y su Influencia en Arquitecturas Modernas

## 6.1. La reutilización en Microservices: un cambio de filosofía

La Service-Oriented Architecture (SOA) con orquestación se fundamentó en una filosofía de reutilización agresiva a nivel empresarial. Richards y Ford (2020) señalan que "los arquitectos en esta arquitectura fueron instruidos para encontrar oportunidades de reutilización de la manera más agresiva posible". Esto conducía a la creación de servicios canónicos compartidos —por ejemplo, un único servicio Customer que todas las divisiones de una aseguradora debían consumir—. Sin embargo, esta aproximación reveló un trade-off fundamental: la reutilización conlleva acoplamiento. Como establece la Primera Ley de la Arquitectura de Software, "todo en arquitectura es un trade-off" (Richards & Ford, 2020). La consolidación de comportamiento en un único lugar genera efectos de ondulación: un cambio en el servicio Customer impacta a todos los servicios que lo consumen, haciendo el cambio riesgoso y exigiendo despliegues coordinados y pruebas holísticas.

Los microservicios invierten esta prioridad. Richards y Ford (2020) afirman que "el objetivo principal de los microservicios es el alto desacoplamiento, modelando físicamente la noción lógica de bounded context". Cuando el arquitecto debe elegir entre reutilización y desacoplamiento, **los microservicios favorecen la duplicación sobre el acoplamiento**.

### 6.1.1. Reutilización de lógica de negocio: duplicación deliberada

En microservicios, la reutilización de lógica de negocio se aborda mediante el concepto de *bounded context* del Domain-Driven Design (DDD). Evans (2003) explica que "el concepto de bounded context reconoce que cada entidad funciona mejor dentro de un contexto localizado. Así, en lugar de crear una clase Customer unificada en toda la organización, cada dominio del problema puede crear la suya propia y reconciliar diferencias en los puntos de integración".

Esto implica que si dos servicios necesitan conceptos similares (por ejemplo, una dirección o un cliente), cada uno define su propia versión adaptada a sus necesidades específicas. La validación de clientes, por ejemplo, no se centraliza en un servicio compartido; cada bounded context implementa las reglas de validación que le son relevantes. Esto evita el acoplamiento estático que surgiría de compartir una definición de clase común.

Richards y Ford (2020) ilustran esto con el ejemplo de la aseguradora: mientras que SOA extraería un servicio Customer canónico que incluya todos los detalles (licencia de conducir para auto, información médica para discapacidad), los microservicios permiten que cada división mantenga su propia representación de cliente, evitando que el equipo de discapacidad tenga que lidiar con la complejidad adicional de una definición única de cliente.

### 6.1.2. Reutilización operativa: el patrón Sidecar y la Service Mesh

La reutilización operativa —concerns como logging, monitoreo, circuit breakers y service discovery— se aborda en microservicios mediante el **patrón sidecar** y la **service mesh**. Richards y Ford (2020) describen:

> "El componente sidecar maneja todos los concerns operativos que los equipos se benefician de acoplar juntos. Así, cuando llega el momento de actualizar la herramienta de monitoreo, el equipo de infraestructura compartida puede actualizar el sidecar, y cada microservicio recibe esa nueva funcionalidad." (Richards & Ford, 2020)

Este enfoque separa claramente la reutilización operativa (que sí se beneficia del acoplamiento) de la reutilización de dominio (que se beneficia del desacoplamiento). La service mesh forma un plano de servicio que conecta todos los sidecars, proporcionando una consola holística para el control operativo global.

En contraste, SOA típicamente incluía servicios de infraestructura (monitoreo, logging, autenticación) como servicios de grano fino dentro de la taxonomía, lo que creaba dependencias operativas acopladas a través del motor de orquestación.

## 6.2. Escenarios donde la orquestación explícita sigue siendo valiosa

A pesar de las desventajas de la orquestación centralizada (cuello de botella, acoplamiento, punto único de fallo), existen escenarios donde un enfoque de orquestación —ya sea un mediador local en microservicios o un motor de orquestación en SOA— puede ser la elección correcta.

### 6.2.1. Flujos de trabajo complejos con múltiples pasos condicionales

Richards y Ford (2020) señalan que la orquestación se usa cuando se requiere control sobre el workflow de un proceso de eventos. En microservicios, cuando un flujo de trabajo empresarial involucra múltiples servicios con lógica condicional compleja, manejo de errores sofisticado y necesidad de estado transaccional, un mediador local (orquestación) puede ser preferible a la coreografía pura.

**Ejemplo: el patrón Saga en microservicios.** El patrón Saga es un caso donde la orquestación explícita es necesaria. Richards y Ford (2020) describen:

> "Un servicio actúa como mediador a través de múltiples llamadas a servicios y coordina la transacción. El mediador llama a cada parte de la transacción, registra éxito o fracaso, y coordina los resultados. Si algo sale mal, el mediador debe asegurar que ninguna parte de la transacción tenga éxito si una parte falla." (Richards & Ford, 2020)

Un ejemplo concreto sería un sistema de reserva de viajes que coordina: (1) reserva de vuelo, (2) reserva de hotel, (3) alquiler de coche y (4) procesamiento de pago. Si el pago falla después de que el vuelo y el hotel han sido reservados, el mediador debe ejecutar transacciones compensatorias (cancelar vuelo, cancelar hotel). Sin un mediador que conozca el estado

completo de la transacción, este flujo sería extremadamente difícil de coordinar mediante coreografía pura.

No obstante, Richards y Ford (2020) advierten: “Algunas transacciones a través de servicios son a veces necesarias; si es la característica dominante de la arquitectura, se cometieron errores”. La orquestación debe usarse con moderación.

### 6.2.2. Flujos de trabajo con intervención humana y aprobaciones

En SOA, los motores de orquestación basados en BPEL (Business Process Execution Language) o BPM (Business Process Management) eran apropiados para flujos que requerían intervención humana. Richards y Ford (2020) describen un escenario de trading donde el mediador de eventos tendría que detener el procesamiento del evento, enviar una notificación a un trader senior para aprobación manual, y esperar a que ocurra esa aprobación.

En microservicios modernos, este tipo de flujo podría implementarse con un servicio mediador local que gestione el estado del workflow y las aprobaciones humanas, similar al patrón de orquestación descrito por Richards y Ford (2020). Un ejemplo sería un proceso de aprobación de préstamos hipotecarios que requiere: verificación de crédito automatizada, revisión manual por un oficial de crédito, aprobación final por un comité y desembolso. La orquestación explícita proporciona visibilidad y control sobre el estado de cada solicitud.

### 6.2.3. Integración con sistemas legacy y terceros

Cuando un flujo de trabajo debe integrar sistemas legacy que no pueden modificarse para participar en coreografía, un mediador de orquestación actúa como un *integration hub*. Richards y Ford (2020) describen cómo el motor de orquestación en SOA actúa como un hub de integración, permitiendo a los arquitectos integrar código personalizado con sistemas de paquetes y software legacy.

En un contexto de microservicios, esto se traduciría en un servicio mediador que encapsula la complejidad de interactuar con sistemas mainframe, SAP o APIs de terceros con protocolos heterogéneos, presentando una interfaz simplificada al resto del ecosistema de microservicios.

### 6.2.4. Procesos con requisitos de auditoría y compliance estrictos

En industrias reguladas (finanzas, salud, seguros), los flujos de trabajo a menudo requieren trazabilidad completa de cada paso, con registros de auditoría inmutables. Un motor de orquestación centralizado facilita este requisito al mantener un log centralizado del estado de cada transacción y cada decisión tomada.

Richards y Ford (2020) mencionan que los arquitectos a menudo incluyen service discovery en la service mesh y que la orquestación puede proporcionar visibilidad y control sobre los flujos. En un contexto regulatorio, esta visibilidad es crítica.

### 6.2.5. El patrón Front Controller en microservicios

Richards y Ford (2020) describen un patrón híbrido llamado *front controller*:

> "El primer servicio llamado debe coordinar a través de una amplia variedad de otros servicios, actuando básicamente como un mediador además de sus otras responsabilidades de dominio. Este patrón se llama el patrón front controller, donde un servicio nominalmente coreografiado se convierte en un mediador más complejo para algún problema." (Richards & Ford, 2020)

Este patrón es útil cuando un servicio específico (por ejemplo, el servicio de OrderPlacement en un e-commerce) naturalmente orquesta varios servicios downstream (pago, inventario, envío) como parte de su responsabilidad principal. La orquestación está localizada en el servicio que ya tiene la responsabilidad del flujo, evitando un mediador global separado.

## 6.3. Trade-offs de la orquestación explícita

Es crucial reconocer que la orquestación, incluso en sus formas modernas y localizadas, introduce trade-offs significativos (Richards & Ford, 2020):

*Cuadro 1: Trade-offs de la topología mediator (orquestación)*

| **Ventaja** | **Desventaja** |
|---|---|
| Control explícito del workflow | Mayor acoplamiento entre servicios |
| Manejo de errores centralizado | Menor escalabilidad (el mediador puede ser cuello de botella) |
| Capacidad de recuperación y reinicio | Menor rendimiento (el mediador añade latencia) |
| Mejor consistencia de datos | Menor tolerancia a fallos (punto único de fallo) |
| Visibilidad y auditoría | Modelado complejo de workflows dinámicos |

Richards y Ford (2020) concluyen que la elección entre las topologías broker y mediator esencialmente se reduce a un trade-off entre control del workflow y capacidad de manejo de errores versus alto rendimiento y escalabilidad.

## 6.4. Conclusión

La SOA legó a las arquitecturas modernas tanto advertencias como principios valiosos. La lección principal —que la reutilización agresiva conduce al acoplamiento y a la fragilidad— fue internalizada por los microservicios, que favorecen la duplicación sobre el acoplamiento y utilizan bounded contexts para aislar la lógica de negocio.

Sin embargo, la orquestación explícita sigue siendo relevante en escenarios específicos: flujos de trabajo complejos con múltiples pasos condicionales, procesos que requieren intervención humana, integración con sistemas legacy y requisitos estrictos de auditoría. En estos casos, un mediador local (en microservicios) o un motor de orquestación (en SOA) proporciona el control, la visibilidad y el manejo de errores que la coreografía pura no puede ofrecer.

La clave está en aplicar la orquestación de manera **quirúrgica y localizada**, no como un principio arquitectónico global. Como advierten Richards y Ford (2020): “No hagas transacciones en microservicios—arregla la granularidad en su lugar”. La orquestación debe ser la excepción, no la regla, y debe implementarse con plena conciencia de sus trade-offs.

## Referencias Bibliográficas


- Evans, E. (2003). *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Addison-Wesley Professional.
- Hohpe, G., & Woolf, B. (2003). *Enterprise Integration Patterns: Designing, Building, and Deploying Messaging Solutions*. Addison-Wesley.
- Richards, M., & Ford, N. (2020). *Fundamentals of Software Architecture: An Engineering Approach*. O'Reilly Media.
- Richardson, C. (2018). *Microservices Patterns*. Manning Publications.

## Declaración de Uso de la Inteligencia Artificial

- **Preguntas 1 y 2:** Para la elaboración de las preguntas 1 y 2, el estudiante Marlon Cuellar utilizó la herramienta ChatGPT (versión GPT-6 Astra) con el propósito de revisar y corregir el uso de citas bibliográficas.
- **Pregunta 3:** Para la elaboración de las respuestas a las pregunta de este seminario se utilizó la herramienta Claude (modelo Claude Opus 3.5, de Anthropic). El integrante Raciel Pupo Santos, responsable de esta pregunta utilizó Claude para mejorar la estructura y redacción de la argumentación a partir de la bibliografía recomendada.
- **Pregunta 4**: el estudiante Ramón Hernández Camacho hizo uso de Gemini para labores de traducción, unificación de los documentos, correcciones de formato, revisión ortográfica; además, la empleó para entender mejor conceptos mientras usaba la bibliografía y en labores de resumen.
- **Pregunta 5:** Para la elaboración de la respuesta a la pregunta de este seminario se utilizó la herramienta Claude (modelo Claude Sonnet 3.5, de Anthropic). El integrante que la empleó fue Leandro Márquez Blanco, el cual elaboró primero una solución propia, que se proporcionó a la herramienta junto con el enunciado del seminario, y Claude se utilizó para mejorar la redacción y la estructura de la respuesta, ampliar la argumentación con la bibliografía recomendada.
- **Pregunta 6:** Para la elaboración de este seminario, el equipo utilizó la herramienta ChatGPT (versión GPT-4) con el propósito de generar ideas iniciales para la estructura del documento, mejorar la redacción de algunos párrafos y apoyar la generación del código LATEX para el formateo del documento. El integrante que hicieron uso de esta herramienta fue Charly Blanco Sánchez Todo el contenido generado fue revisado, verificado y adaptado por el equipo, quien asume la responsabilidad final del documento.